



4EYES Labs Audit

Machina Finance: Grid Trading

Auditor: Luca D'Angelo

Date: 2026-03-05



Table of Contents

Table of Contents	2
Disclaimer	3
Audit Summary	4
Contracts	5
E2T Grid Order	6
T2T Grid Order	11
E2T Limit Order	16
Appendix	21
Issue Guide	22
About Us	22

Disclaimer

This audit is provided as-is at the time of its completion and might not reflect the most up-to-date state of any smart-contracts deployed, or any design document, or any research paper referenced herein.

Audit Summary

Protocol	Machina Finance: Grid Trading
Repository	https://github.com/machinafi/sdk
Commit	b9421b57ae7bdcf34f7f8405a839185fa4c82f

Contract	File Name	Severity	Status
E2T Grid Order	e2t-grid-order.es	WARNING	RESOLVED
T2T Grid Order	t2t-grid-order.es	WARNING	RESOLVED
E2T Limit Order	e2t-limit-order.es	WARNING	RESOLVED

Contracts

E2T Grid Order

Description

- The contract defines an ERG-2-Token grid order where the following actions are expected: buy, sell, close.
- The close condition is just defined as the grid creator being able to spend the box however they want.

Box Contents

Registers

- The order creator script as a SigmaProp, in register R4.
- The buy and sell prices in a Coll[Long], in register R5. Where prices are defined in units of nanoergs per quote token amount. If token decimals are required, then it can be found by looking at the EIP4 minting transaction.
- The box id of the parent E2T grid order as a Coll[Byte], in register R6. This would be empty when the order is initially created.

Tokens

- Index 0: The quote token meant to be bought or sold for the grid order. It is possible that there are no quote tokens remaining in the box, e.g. if all tokens are bought.

Context Variables

- Index 0: A Boolean value such that true represents a buy order and false represents a sell order.
- Index 1: An Int value representing the output index of the recreated grid order box.

Compile-Time Constants

- None

Contract Analysis

- L24-L33 defines global variables used in the remaining contract.
 - The token index in the grid order box, at index 0.
 - The buy price index in the prices collection, at index 0.
 - The sell price index in the prices collection, at index 1.
 - L28 defines the TOKEN_ID as a hard-coded constant.
 - Will this be a compile-time constant or stored in a register?
- L35 If an output grid order is present, so a trade occurred, then:
 - L40-L47

4EYES Labs

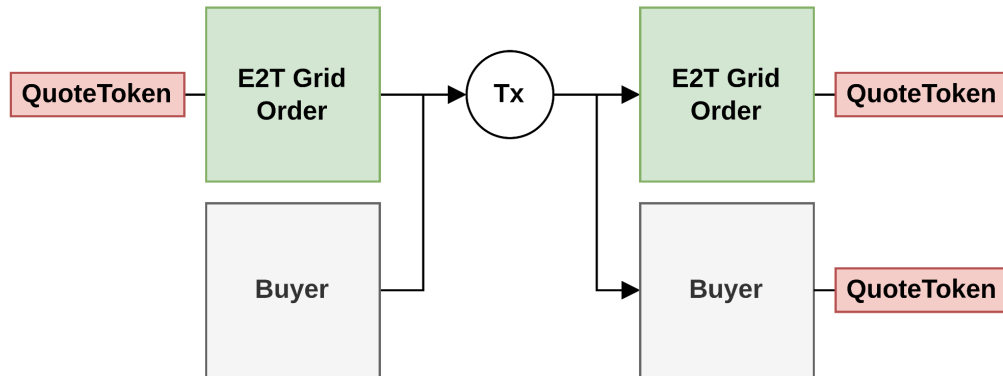
- Determines the order action from the context variable index 0, either buy or sell.
- Obtains the prices collection from register R5.
- The childBox grid order output from its expected index provided as the context variable at index 1.
- The expected quote token and the expected output quote token, if not empty. These should be at the same index in the tokens collection of each respective box.
- L49-L54 defines validRecreation, which checks that the output grid order box is recreated properly:
 - It must maintain the same ErogTree.
 - The output token id at index 0 must be the same as the input token id at index 0.
 - It must preserve the same owner SigmaProp in register R4.
 - It must preserve the price collection in register R5.
 - It stores the input order box id SELF.id in register R6.
- L56-L88 defines validExchange, which checks that either a buy or sell order was correctly executed:
 - L57 if a buy order is meant to be executed, where the buyer wants to acquire tokens by providing nanoergs, then:
 - L62 defines price as the BUY price is obtained from the price collection.
 - L63 defines nanoergsIn as the nanoergs provided to the grid order box, calculated as the difference of the output order box and the input order box value, since the output order box should have an increase in nanoergs.
 - L64 defines tokensOut as the tokens received from the grid order box, calculated as the difference of the input order box and the output order box token amounts, since the output order box would have a decrease in the quote token amount.
 - L65 defines requiredNanoergs as the product of tokensOut and price. Since prices are defined in units of nanoergs per quote token, the units will cancel out to provide a value in nanoergs only. This value represents the exact amount that should be provided by the buyer based on the token price and the tokens emitted from the order box.
 - L67-L69 defines validBuy where the following is checked:
 - L68 the amount of nanoergs provided by the buyer is positive.
 - L69 the amount of nanoergs provided by the buyer is at least the exact amount expected from the token price and the amount emitted from the order box.
 - Otherwise, a sell order is meant to be executed, where the seller wants to acquire nanoergs by providing tokens, then:

4EYES Labs

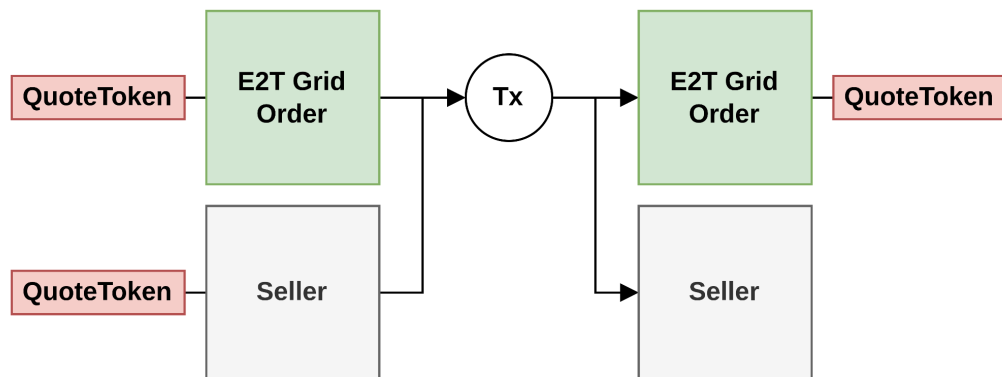
- The SELL price is obtained from the prices collection.
- The difference in nanoergs from the input order box and the output order box is calculated. Since nanoergs will be emitted, the input order box will contain more nanoergs than the output.
- The amount of tokens provided by the seller is calculated as the difference of the tokens in the output order box and the input order box, since the output will have an increase in token amount.
- The minimum amount of nanoergs provided to the seller from the order box is calculated as the product of the token price and the tokens provided by the seller.
- L83-L85 defines validSell where the following is checked:
 - The token amount provided by the seller must be positive.
 - The amount of nanoergs provided by the order box to the seller must be at most the amount of nanoergs determined from the tokens provided by the seller. This ensures more nanoergs are not emitted from the order box than there needs to be.
- If the sell empties the nanoergs in the box, then the box min nanoerg condition will fail, so the order/tx will fail.
 - Does this mean there is an implicit max order size to prevent this?
 - If so, should this be calculated since the min nanoerg per byte is known?
 - Or is this handled in the frontend somehow?
- L90 checks that both validRecreation and validExchange are true.
- Otherwise, the grid order creator can only spend the box.

Transaction Diagram

E2T Grid Order: Buy



E2T Grid Order: Sell



Review Comments

1. The constant will not be stored in the register. The magic byte sequence in the ErgoTree template will be replaced with the correct token id bytes.
2. This was done on purpose as a contract optimization. Since the blockchain will guarantee a transaction failure in this scenario, it can be handled by off-chain code deterministically.

T2T Grid Order

Description

- The contract defines a Token-2-Token grid order where the following actions are expected: buy, sell, close.
- The close condition is just defined as the grid creator being able to spend the box however they want.

Box Contents

Registers

- The order creator script as a SigmaProp, in register R4.
- The buy and sell prices in a Coll[Long], in register R5. Where prices are defined in units of base token per quote token amount. If token decimals are required, then it can be found by looking at the EIP4 minting transaction of the base and quote tokens.
- The box id of the parent T2T grid order as a Coll[Byte], in register R6. This would be empty when the order is initially created.

Tokens

- Index 0: The base token representing the asset used to purchase the quote token and the asset received when selling the quote token. To ensure that token ordering is preserved, it is expected that there is at least one unit of base token when the order is created, i.e. the base token cannot be empty.
- Index 1: The quote token meant to be bought or sold for the grid order. It is possible that there are no quote tokens remaining in the box, e.g. if all quote tokens are bought.

Context Variables

- Index 0: A Boolean value such that true represents a buy order and false represents a sell order.
- Index 1: An Int value representing the output index of the recreated grid order box.

Compile-Time Constants

- None

Contract Analysis

- L25-L30 defines global variables used in the remaining contract.
 - The base token index in the grid order box, at index 0.
 - The quote token index in the grid order box, at index 1.
 - The buy price index in the prices collection, at index 0.
 - The sell price index in the prices collection, at index 1.

4EYES Labs

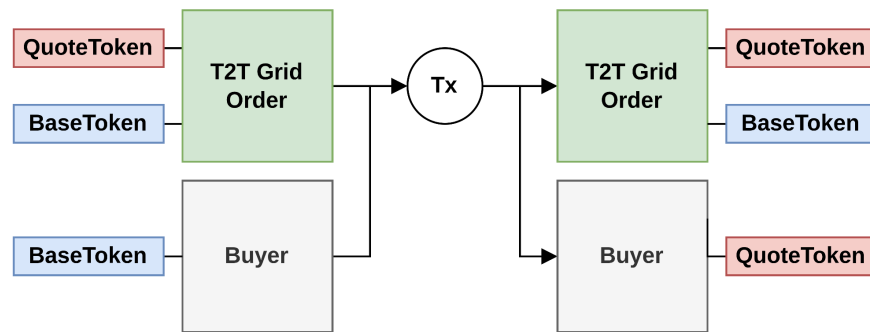
- L28 defines the QUOTE_TOKEN_ID as a hard-coded constant.
 - Will this be a compile-time constant or stored in a register?
- L37 If an output grid order is present, so a trade occurred, then:
 - L42-L55
 - Determines the order action from the context variable index 0, either buy or sell.
 - Obtains the prices collection from register R5.
 - The childBox grid order output from its expected index provided as the context variable at index 1.
 - The expected base token and the expected output base token. These should be at the same index in the tokens collection of each respective box.
 - The expected quote token and the expected output quote token, if not empty. These should be at the same index in the tokens collection of each respective box.
 - The expected base token amounts and quote token amounts from the input grid order box.
 - The expected base token amounts and quote token amounts from the output grid order box.
 - L57-L64 defines validRecreation, which checks that the output grid order box is recreated properly:
 - It must preserve the same ErogTree.
 - It must preserve the same nanoergs value in the box since base tokens are being traded for quote tokens, not nanoergs.
 - The output base token id at index 0 must be the same as the input base token id at index 0. This will always be present in the output grid order box.
 - The output quote token id must be the same quote token id referenced in the input grid order box.
 - For the buying action, if all quote tokens are purchased, then the output will be empty and this check will fail.
 - It must preserve the same owner SigmaProp in register R4.
 - It must preserve the price collection in register R5.
 - It stores the input order box id SELF.id in register R6.
 - L66-L98 defines validExchange, which checks that either a buy or sell order was correctly executed:
 - L66 if a buy order is meant to be executed, where the buyer wants to acquire tokens by providing nanoergs, then:
 - L72 defines price as the BUY price is obtained from the price collection.
 - L73 defines baseIn as the base tokens provided to the grid order box, calculated as the difference of the output order box and the input order box base token amounts, since the output order box should have an increase in base tokens.

4EYES Labs

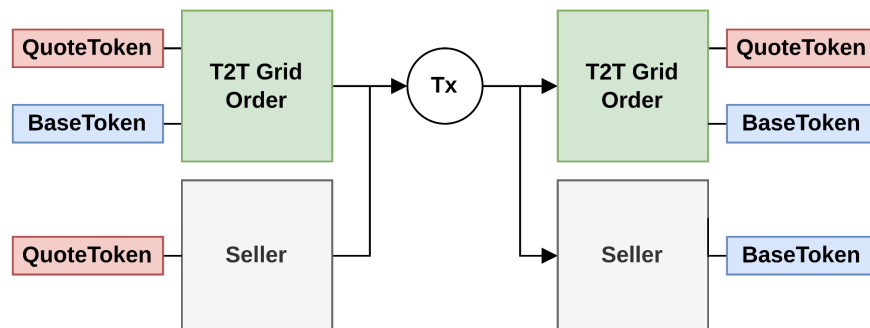
- L64 defines quoteOut as the quote tokens received from the grid order box, calculated as the difference of the input order box and the output order box quote token amounts, since the output order box would have a decrease in the quote token amount.
- L65 defines requiredBaseAmount as the product of quoteOut and price. Since prices are defined in units of base tokens per quote token, the units will cancel out to provide a value in base tokens only. This value represents the exact amount that should be provided by the buyer based on the quote token price and the quote tokens emitted from the order box.
- L77-L79 defines validBuy where the following is checked:
 - L78 the amount of base tokens provided by the buyer is positive.
 - L69 the amount of base tokens provided by the buyer is at least the exact amount expected from the quote token price and the quote token amount emitted from the order box.
- Otherwise, a sell order is meant to be executed, where the seller wants to acquire base tokens by providing quote tokens, then:
 - The SELL price is obtained from the prices collection.
 - The difference in base tokens from the input order box and the output order box is calculated. Since base tokens will be emitted, the input order box will contain more base tokens than the output.
 - The amount of quote tokens provided by the seller is calculated as the difference of the tokens in the output order box and the input order box, since the output will have an increase in quote token amount.
 - The minimum amount of base tokens provided to the seller from the order box is calculated as the product of the quote token price and the quote tokens provided by the seller.
 - L93-L95 defines validSell where the following is checked:
 - The token amount provided by the seller must be positive.
 - The amount of base tokens provided by the order box to the seller must be at most the amount of base tokens determined from the quote tokens provided by the seller. This ensures more base tokens are not emitted from the order box than there needs to be.
 - L90 checks that both validRecreation and validExchange are true.
- Otherwise, the grid order creator can only spend the box.

Transaction Diagram

T2T Grid Order: Buy



T2T Grid Order: Sell



Review Comments

1. The constant will not be stored in the register. The magic byte sequence in the ErgoTree template will be replaced with the correct token id bytes.
2. In fact, the check will not fail. It will fallback onto the `EMPTY_QUOTE_TOKEN` which has the same token id as the quote token id. This was missed in the initial audit.

E2T Limit Order

Description

- The contract defines an ERG-2-Token grid order where the following actions are expected: buy, sell, close.
- The close condition is just defined as the grid creator being able to spend the box however they want.

Box Contents

Registers

- The order creator script as a SigmaProp, in register R4. Due to how the rest of the contract is defined, only functionality for ErgoTree v0 is supported.
- The limit price for either a buy or sell order as a Long, in register R5. Where the price is defined in units of nanoergs per quote token amount. If token decimals are required, then it can be found by looking at the EIP4 minting transaction.
- The order action type, as a Boolean, in register R6, where true is buy and false is sell.
- The box id of the parent E2T limit order as a Coll[Byte], in register R7. This would be empty when the order is initially created.

Tokens

- Index 0: The quote token meant to be bought or sold for the limit order. It is possible that there are no quote tokens remaining in the box, e.g. if all tokens are bought.

Context Variables

- Index 0: An Int value representing the output index of the recreated limit order box.

Compile-Time Constants

- None

Contract Analysis

- L25-L35 defines global variables used in the remaining contract.
 - The token index in the limit order box, at index 0.
 - L26 defines the `TOKEN_ID` as a hard-coded constant.
 - Will this be a compile-time constant or stored in a register?
 - An empty token placeholder using the quote token id. Empty quote tokens can happen for a buy order that is fully fulfilled, or a sell order when it is initially created.

4EYES Labs

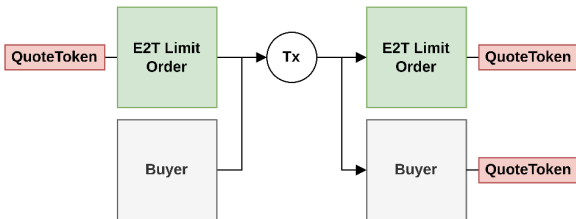
- A constant to be used representing the minimum nanoerg expected in the limit order box.
- The owner script from R4.
- The price from R5.
- The limit order action from R6.
- The output index of the output limit order box.
- A trading flag defined as true if the output limit order box exists.
- L37 If an output limit order is present, so a trade occurred, then:
 - L38-L43
 - The childBox grid order output from its expected index provided as the context variable at index 0.
 - The expected quote token and the expected output quote token, if not empty. These should be at the same index in the tokens collection of each respective box.
 - The expected quote token and the expected output quote token amounts.
 - L45 Defines isPartial as a flag to check if the limit order is a partial buy or a partial sell order, where:
 - If the limit order is created as a buy order, then:
 - The amount of tokens in the output limit order box should be positive, since not all of the tokens were purchased by the buyer.
 - Otherwise, the amount of nanoergs in the output limit order box should be greater than or equal to the predefined minimum nanoerg amount expected, since not all of the nanoerg was given to the seller.
 - L49-L66 defines validRecreation, which checks that the output grid order box is recreated properly:
 - If the limit order is a partial order, then:
 - It must maintain the same ErogTree.
 - The output quote token id at index 0 must be the same as the input quote token id at index 0.
 - It must preserve the same owner SigmaProp in register R4.
 - It must preserve the price in register R5
 - It must preserve the order type in register R6.
 - It stores the input order box id SELF.id in register R7.
 - Otherwise, the order is fully fulfilled, i.e. all tokens are purchased or all nanoergs are released from selling, where:
 - The output limit order box contract is made to match the order creator address, thus the funds (nanoerg from buy, quote tokens from sell) are sent to the order owner.
 - The output limit order box, now belonging to the owner, stores the input order box id SELF.id in register R4.
 - Why is this necessary, since in this case the order is no longer active?
 - L68-L98 defines validExchange, which checks that either a buy or sell limit order was correctly executed:

4EYES Labs

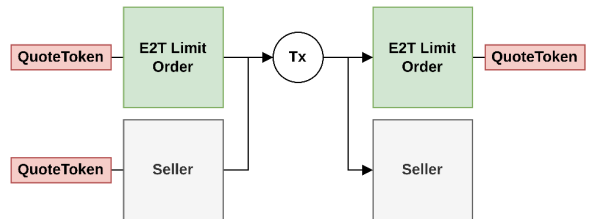
- L68 if a buy order is meant to be executed, where the buyer wants to acquire quote tokens by providing nanoergs, then:
 - L74 defines nanoergsIn as the nanoergs provided to the grid order box, calculated as the difference of the output order box and the input order box value, since the output order box should have an increase in nanoergs.
 - L75 defines tokensOut as the tokens received from the grid order box, calculated as the difference of the input order box and the output order box token amounts, since the output order box would have a decrease in the quote token amount.
 - L76 defines requiredNanoergs as the product of tokensOut and price. Since prices are defined in units of nanoergs per quote token, the units will cancel out to provide a value in nanoergs only. This value represents the exact amount that should be provided by the buyer based on the token price and the tokens emitted from the limit order box.
 - L78-L80 defines validBuy where the following is checked:
 - The amount of nanoergs provided by the buyer is positive.
 - The amount of nanoergs provided by the buyer is at least the exact amount expected from the token price and the amount emitted from the order box.
- Otherwise, a sell order is meant to be executed, where the seller wants to acquire nanoergs by providing quote tokens, then:
 - The difference in nanoergs from the input order box and the output order box is calculated. Since nanoergs will be emitted, the input order box will contain more nanoergs than the output.
 - The amount of tokens provided by the seller is calculated as the difference of the tokens in the output order box and the input order box, since the output will have an increase in token amount.
 - The minimum amount of nanoergs provided to the seller from the order box is calculated as the product of the token price and the tokens provided by the seller.
 - L93-L95 defines validSell where the following is checked:
 - The token amount provided by the seller must be positive.
 - The amount of nanoergs provided by the order box to the seller must be at most the amount of nanoergs determined from the tokens provided by the seller. This ensures more nanoergs are not emitted from the order box than there needs to be.
 - L100 checks that both validRecreation and validExchange are true.
- Otherwise, the grid order creator can only spend the box.

Transaction Diagram

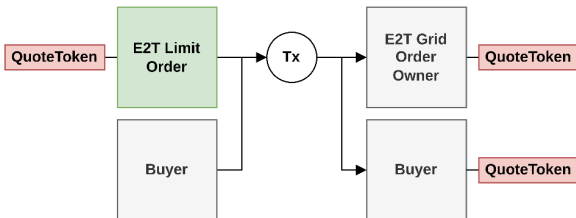
E2T Partial Limit Order: Buy



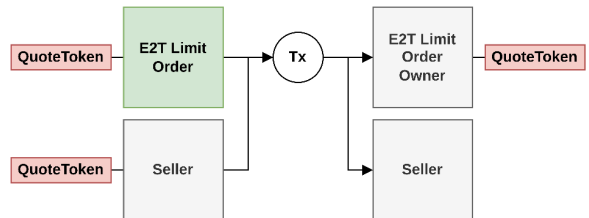
E2T Partial Limit Order: Sell



E2T Full Limit Order: Buy



E2T Full Limit Order: Sell



Review Comments

1. The constant will not be stored in the register. The magic byte sequence in the ErgoTree template will be replaced with the correct token id bytes.
2. This is done on purpose to ensure that the correct amounts of an order go to the correct utxo in case there are multiple orders from the same owner in one transaction.

Appendix

Issue Guide

Severity	Description
CRITICAL	The contract contains one or more serious errors and warnings.
SERIOUS	An error that will cause unexpected behavior.
WARNING	A property that is questionable but will not cause unexpected behavior.
GOOD	Everything works as expected.

Status	Description
RESOLVED	The issues have been addressed. Default status if severity is GOOD.
UNRESOLVED	The issues have not been addressed.

About Us

4EYES Labs is an ErgoScript design and auditing firm for Ergo blockchain.

Linktree: <https://linktr.ee/4EYESLabs>